
GUÍA BREVE PARA ESTUDIANTES

Crear un dashboard con resultados ya modelados

Plantilla sencilla en Dash y Plotly

Flujo: exportar CSV, validar y ejecutar

Esta versión no genera datos ni entrena modelos. El estudiante entrega al dashboard los resultados obtenidos previamente en su propio análisis.

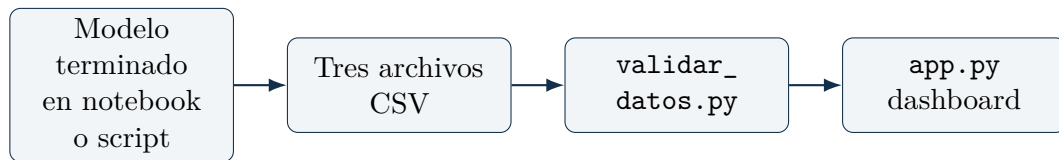
Contenido

1	Qué hace esta plantilla	2
2	Archivos que recibe el estudiante	2
3	Paso 1. Finalizar el modelo	2
4	Paso 2. Exportar los tres CSV	3
4.1	Archivo 1: predicciones.csv	3
4.2	Archivo 2: metricas_modelos.csv	3
4.3	Archivo 3: importancia_variables.csv	3
5	Paso 3. Reemplazar los datos de ejemplo	3
6	Paso 4. Personalizar los textos	4
7	Paso 5. Preparar el entorno	4
7.1	Windows PowerShell	4
7.2	Linux, macOS, Jupyter o Google Colab	4
8	Paso 6. Validar los datos	4
9	Paso 7. Ejecutar el dashboard	5
10	Paso 8. Leer el dashboard	5
11	Errores frecuentes	6
12	Lista final de comprobación	6

1 Qué hace esta plantilla

La plantilla convierte los resultados de un modelo de clasificación binaria en un dashboard interactivo. El estudiante puede usar regresión logística, árboles, bosques, redes neuronales u otro algoritmo. El dashboard no necesita conocer cómo se entrenó el modelo.

Solo recibe tres archivos CSV:



Diferencia frente al caso avanzado. El proyecto avanzado incluye generación sintética, procesamiento, entrenamiento, iteraciones y serialización del modelo. Esta plantilla comienza cuando ese trabajo ya terminó.

2 Archivos que recibe el estudiante

Al descomprimir el ZIP aparece la carpeta `Plantilla_Dashboard_Resultados_Modelados`:

Archivo	Uso
<code>app.py</code>	Construye el dashboard, sus gráficos, filtros y callbacks.
<code>config.py</code>	Permite cambiar títulos, nombres y colores sin editar <code>app.py</code> .
<code>data_utils.py</code>	Lee los CSV y verifica columnas, tipos y rangos.
<code>validar_datos.py</code>	Ejecuta la validación antes de abrir el dashboard.
<code>data/</code>	Contiene los tres CSV que el estudiante debe reemplazar.
<code>assets/style.css</code>	Controla la apariencia del dashboard.
<code>FORMATO_DATOS.md</code>	Describe el contrato completo de los CSV.
<code>requirements.txt</code>	Lista las bibliotecas necesarias.

3 Paso 1. Finalizar el modelo

Antes de utilizar la plantilla, el estudiante debe tener:

- un conjunto de prueba que el modelo no utilizó para entrenarse;
- la clase real de cada observación, codificada como 0 o 1;
- la probabilidad estimada de la clase positiva;
- las métricas de uno o varios modelos;
- una medida de importancia de variables.

La plantilla está diseñada para clasificación binaria. Por ejemplo: abandona/no abandona, fraude/no fraude, compra/no compra o falla/no falla.

4 Paso 2. Exportar los tres CSV

4.1 Archivo 1: predicciones.csv

Cada fila representa una observación evaluada.

Columna	Tipo	Contenido
id	texto	Identificador único y sin datos personales sensibles.
grupo	texto	Categoría que se desea comparar, como programa, sede o producto.
periodo	texto	Cohorte, mes, semestre o periodo de análisis.
valor_real	0 o 1	Clase observada.
probabilidad	0 a 1	Probabilidad predicha para la clase positiva.

Se pueden conservar columnas adicionales. Estas aparecerán en la tabla del dashboard.

Ejemplo de exportación desde un notebook:

```
salida = X_prueba.copy()
salida["id"] = ids_prueba
salida["grupo"] = grupos_prueba
salida["periodo"] = periodos_prueba
salida["valor_real"] = y_prueba
salida["probabilidad"] = modelo.predict_proba(X_prueba)[: , 1]
salida.to_csv("data/predicciones.csv", index=False)
```

4.2 Archivo 2: metricas_modelos.csv

Cada fila representa un modelo o una iteración. Las columnas obligatorias son:

```
modelo,exactitud,precision,recall,f1,roc_auc
Regresion logistica,0.82,0.71,0.68,0.69,0.86
Bosque aleatorio,0.85,0.76,0.72,0.74,0.89
```

Las métricas deben escribirse como decimales entre 0 y 1. Por ejemplo, 85% se escribe como 0.85.

4.3 Archivo 3: importancia_variables.csv

Debe contener dos columnas:

```
variable,importancia
asistencia,0.18
promedio,0.13
edad,0.04
```

El informe del proyecto debe explicar si la importancia corresponde a coeficientes, permutación, SHAP u otro método.

5 Paso 3. Reemplazar los datos de ejemplo

Abra la carpeta data/. Allí encontrará tres archivos de ejemplo:

- predicciones.csv;
- metricas_modelos.csv;

- `importancia_variables.csv`.

Reemplácelos por sus propios archivos. Mantenga exactamente esos nombres y las columnas obligatorias. No agregue una columna de índice al exportar. En pandas use siempre `index=False`.

La subcarpeta `data/plantillas_vacias/` contiene archivos que solo tienen los encabezados obligatorios. Puede copiarlos y completarlos si prefiere comenzar sin los datos de ejemplo.

Regla práctica. Primero haga funcionar la plantilla con los archivos de ejemplo. Después sustituya un CSV a la vez y ejecute la validación. Así podrá identificar con rapidez qué archivo presenta un problema.

6 Paso 4. Personalizar los textos

Abra `config.py`. Cambie únicamente los valores ubicados a la derecha del signo igual:

```
TITULO = "Riesgo de abandono de clientes"
SUBTITULO = "Resultados del modelo predictivo"
NOMBRE_REGISTRO = "Clientes"
NOMBRE_CLASE_POSITIVA = "Abandono observado"
NOMBRE_GRUPO = "Segmento"
NOMBRE_PERIODO = "Mes"
```

No es necesario modificar `app.py` para cambiar el tema del proyecto.

7 Paso 5. Preparar el entorno

Abra una terminal dentro de la carpeta de la plantilla.

7.1 Windows PowerShell

```
python -m venv .env
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

7.2 Linux, macOS, Jupyter o Google Colab

```
python -m venv .env
source .venv/bin/activate
python -m pip install -r requirements.txt
```

En Jupyter o Colab puede instalar las dependencias con:

```
!python -m pip install -r requirements.txt
```

Para trabajar con mayor facilidad se recomienda ejecutar el dashboard desde una terminal de Visual Studio Code, JupyterLab o el sistema operativo.

8 Paso 6. Validar los datos

Ejecute:

```
python validar_datos.py
```

Si los archivos son correctos aparecerá un mensaje similar a:

```
VALIDACION CORRECTA
Predicciones: 200 filas
Modelos comparados: 3
Variables con importancia: 15
```

Si aparece un error, corríjalo antes de continuar. El validador detecta archivos ausentes, columnas faltantes, archivos vacíos, valores reales diferentes de 0 y 1, probabilidades fuera del intervalo de 0 a 1 y métricas no numéricas.

9 Paso 7. Ejecutar el dashboard

Ejecute:

```
python app.py
```

Abra en el navegador:

<http://127.0.0.1:8050>

También puede usar:

- Windows: doble clic en `ejecutar_dashboard.bat`;
- Linux o macOS: `./ejecutar_dashboard.sh`.

Detenga el servidor con `Ctrl+C` en la terminal.

10 Paso 8. Leer el dashboard

Elemento	Interpretación
Filtros	Restringen los registros por grupo y periodo.
Umbral	Cambia la regla que convierte la probabilidad en clase 0 o 1. No reentrena el modelo.
Tarjetas	Resumen cantidad de registros, clase positiva, predicción positiva y ROC-AUC.
Matriz de confusión	Muestra aciertos y errores para el umbral seleccionado.
Comparación de modelos	Presenta las métricas exportadas por el estudiante.
Importancia	Ordena las variables según el método usado en el análisis.
Tabla	Muestra los casos con mayor probabilidad y permite ordenar o filtrar.

Si un filtro deja una sola clase real, el ROC-AUC aparecerá como `--`. Esto es correcto: esa métrica necesita observaciones de ambas clases.

11 Errores frecuentes

Mensaje o situación	Solución
No se encontró un CSV	Compruebe el nombre y que el archivo esté dentro de <code>data/</code> .
Faltan columnas	Compare el encabezado con <code>FORMATO_DATOS.md</code> .
Probabilidad fuera de 0 y 1	Exporte probabilidades, no porcentajes de 0 a 100 ni clases predichas.
Métricas mayores que 1	Convierta los porcentajes a decimales.
No abre el navegador	Copie manualmente http://127.0.0.1:8050 .
El puerto 8050 está ocupado	Cierre otra ejecución del dashboard o cambie el puerto al final de <code>app.py</code> .

12 Lista final de comprobación

Antes de entregar:

- ☐ El modelo fue evaluado con datos no usados durante el entrenamiento.
- ☐ Los tres CSV tienen los nombres y columnas requeridos.
- ☐ El comando `validar_datos.py` termina sin errores.
- ☐ Los títulos de `config.py` describen el proyecto.
- ☐ Los filtros, el umbral y las pestañas funcionan.
- ☐ Las métricas del dashboard coinciden con el análisis del estudiante.
- ☐ El informe explica el significado y las limitaciones de los resultados.

Orden mínimo de trabajo: modelar los datos, exportar los tres CSV, copiarlos en `data/`, editar `config.py`, ejecutar `validar_datos.py` y finalmente ejecutar `app.py`.